

LABORATORY

LAB 17

SD Card Data Logger

Logging ADC Data to an SD Card with EduFramework

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	Data Logging and Data Storage	2
2.2	SD Card in an Embedded System	3
2.3	Filesystem and File Lifecycle	3
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Pin Mapping	4
3.3	Circuit Connections	4
4	EduFramework API	5
4.1	SD_File_t	5
4.2	SD_Begin()	5
4.3	SD_Open() and SD_FILE_WRITE	6
4.4	SD_WriteLine()	6
4.5	SD_Close() and SD_End()	7
5	Lab Exercise	7
5.1	Requirements	7
5.2	Program	7
5.3	Verification	9
6	Extension	10
6.1	Extension Exercise	10
7	References	10

1 Introduction

1.1 Lab Overview

In measurement and monitoring systems, data is not always observed only while an application is running. A data logger acquires data from sensors or other input signals, processes the data with a microcontroller, and stores it so that it can be retrieved or analyzed later. Microcontroller-based data loggers using microSD storage have been applied to analog signal acquisition and long-term monitoring systems [1] [2] .

This lab uses a potentiometer as an analog data source. The S32K144 reads the ADC value and stores ten samples in a file on an SD Card. Serial Monitor is used to observe the sampling process, while the data stored on the card can be checked after the logging operation has completed.

The lab focuses on application-level storage operations. Low-level SPI communication and SD protocol handling are not implemented in the main application; EduFramework provides the SD Card Device API for initialization, file opening, data writing, and resource closing.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the basic data flow of a data logging system that uses a microcontroller and external storage.
2. Explain the roles of the SD Card and filesystem in storing data in an embedded application.
3. Connect the MicroSD module and potentiometer to the MaaZEDU Development Board according to the lab configuration.
4. Use `SD_Begin()` , `SD_Open()` , `SD_WriteLine()` , `SD_Close()` , and `SD_End()` to create a file and write data to it.
5. Verify that ADC data has been stored on the SD Card after the logging operation is complete.

2 Background

2.1 Data Logging and Data Storage

Data logging is the process of acquiring data over time and storing it for later observation or analysis. A system may receive data from sensors, analog signals, or measurement devices, then use a microcontroller as the acquisition and processing unit before writing the data to storage [1] .

A general model can be represented as:

Data Source → Acquisition → Microcontroller → Storage .

In this lab, the potentiometer generates an analog voltage and `analogRead()` converts that signal into a raw ADC value. The value is stored in a file on the SD Card instead of being shown only on Serial Monitor:

Potentiometer → ADC0_SE12 → S32K144 → SD Card → File .

This arrangement demonstrates the difference between observing data in real time and storing data for later use. Data loggers that use removable storage such as microSD are especially useful when a system must retain many measurements without depending on a continuous connection to a computer [2] .

2.2 SD Card in an Embedded System

An SD Card is a flash-memory storage device that can be accessed by a host using communication modes defined by the SD Association [3]. In EduFramework, the SD Card Device uses the framework's existing SPI interface, while the application works only with the higher-level storage APIs [6].

The application in this lab does not need to send SD Card commands directly or manage physical blocks. These operations are handled internally by the SD Card Device. At application level, the objective is simply to initialize the storage device, work with a file, and end the storage session in the correct sequence.

Note

SPI has already been used in previous labs. Lab 17 does not repeat the SPI operating principles; it only uses the `SCK`, `MOSI`, `MISO`, and chip-select signals required to connect the MicroSD module.

2.3 Filesystem and File Lifecycle

A storage device provides physical data space, while a filesystem organizes that space into objects that applications can manipulate, such as files and directories. FatFs provides operations such as mounting a filesystem, opening files, reading, writing, synchronizing, and closing files for embedded systems [4].

EduFramework uses FatFs internally in the SD Card Device but hides its internal structures from the application. After `SD_Begin()` succeeds, the SD Card has been initialized and the filesystem has been mounted. The application can then open a file and write data using EduFramework APIs [6].

The file lifecycle in this lab is limited to four main steps:

`Initialize` → `Open` → `Write` → `Close`.

Closing the file after writing is an important step. FatFs documentation recommends closing an open file after an access session; cached file and filesystem information may not be safely updated if power is removed or the storage medium is detached while the file remains open [4].

Note

Remove the SD Card only after the program has finished writing and closed the file. In this lab, `Logging complete.` is printed after `SD_Close()` and `SD_End()` to indicate that the storage sequence has finished.

3 Hardware Setup

3.1 Required Hardware

This lab uses the MaaZEDU Development Board with an S32K144 microcontroller from the S32K1xx family [5]. The MicroSD module acts as external storage, while the potentiometer provides a variable analog signal for generating test data.

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
MicroSD Card Module	Storage module used to connect an SD Card to the MaaZEDU SPI interface.
MicroSD Card	Storage medium used to save the lab data.
Potentiometer	Provides a variable analog voltage for generating ADC samples.
Jumper wires	Connect power, SPI, chip-select, and the analog signal.
USB Cable	Connects the board to the computer for power, programming, and Serial Monitor.

Table 17.3.1. Hardware used in the lab

3.2 Pin Mapping

The SD Card Device uses the EduFramework SPI signals for clock and data. `GPIO4` is selected as the software chip-select for the MicroSD module. The potentiometer is read through `ADC0_SE12`. The MaaZEDU Guide defines the corresponding SPI and GPIO pins, while EduFramework provides the logical pin names used in application code [7] [6].

Component	Logical Pin	MCU Pin	Function
MicroSD <code>SCK</code>	<code>SPI_SCK</code>	<code>PTB14</code>	SPI Clock
MicroSD <code>MOSI</code>	<code>SPI_SOUT</code>	<code>PTB16</code>	SPI Data: MaaZEDU → MicroSD
MicroSD <code>MISO</code>	<code>SPI_SIN</code>	<code>PTB15</code>	SPI Data: MicroSD → MaaZEDU
MicroSD <code>CS</code>	<code>GPIO4</code>	<code>PTD12</code>	Software Chip-Select
Potentiometer <code>OUT</code>	<code>ADC0_SE12</code>	<code>PTC14</code>	Analog Input

Table 17.3.2. Signal mapping used in Lab 17

3.3 Circuit Connections

The MicroSD module is powered from `3V3` in the hardware configuration verified for this lab. `SCK`, `MOSI`, and `MISO` are connected to the corresponding SPI signals, while `CS` is connected to `GPIO4`. The two outer potentiometer terminals are connected to `3V3` and `GND`; the `OUT` signal terminal is connected to `ADC0_SE12`.

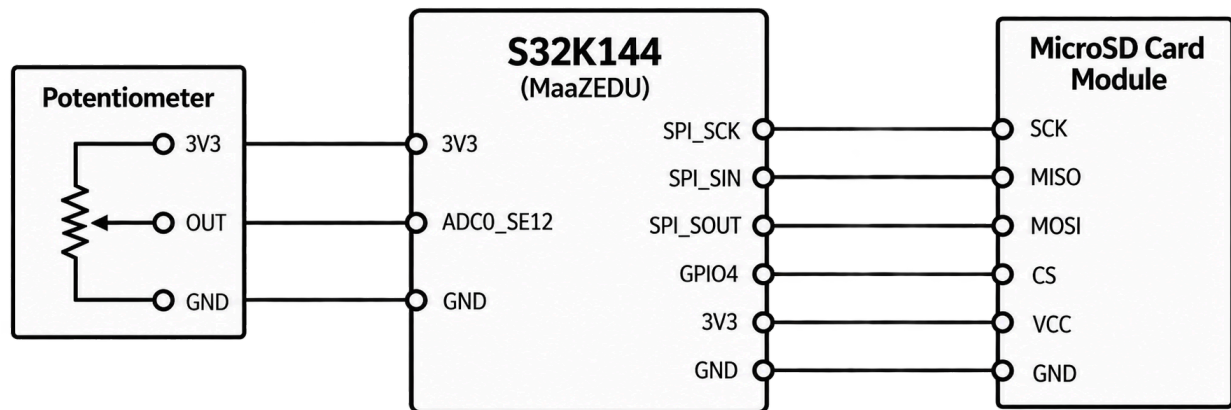


Figure 17.3.1. Connection diagram for the MicroSD module and potentiometer with the MaaZEDU Development Board

Note

Do not remove the SD Card while the program is writing data. After Serial Monitor displays `Logging complete.`, the file has been closed and the SD Device session used in the lab has ended.

4 EduFramework API

The main exercise uses the SD Card Device API to manage the complete storage sequence. Details such as SD commands, block access, and FatFs objects do not appear in the application. The application only keeps an `SD_File_t` and calls APIs corresponding to initialization, opening, writing, and closing a file [6].

4.1 SD_File_t

`SD_File_t` is a file handle managed by the SD Card Device. It is passed to file operation APIs after `SD_Open()` succeeds. The application does not need to access or modify the internal `handle` field directly [6].

Example:

```
SD_File_t file = {SD_INVALID_HANDLE};
```

4.2 SD_Begin()

SD_Begin

Initializes the SD Card through SPI and mounts the filesystem using the selected software chip-select pin.

Syntax

```
SD_Begin(csPin);
```

Parameters

Parameter	Type / accepted value	Description
<code>csPin</code>	Digital Logical Pin	SD Card chip-select pin. This lab uses <code>GPI04</code> .

Return value

`true` if the SD Card is initialized and the filesystem is mounted successfully; `false` if initialization or mounting fails.

In this lab, one call to `SD_Begin(GPI04)` prepares both the SD Card interface and the filesystem before the file is opened [6].

4.3 `SD_Open()` and `SD_FILE_WRITE`**`SD_Open`**

Opens a file and assigns its file handle to an `SD_File_t` variable.

Syntax

```
SD_Open(&file, path, mode);
```

Parameters

Parameter	Type / accepted value	Description
<code>file</code>	SD File	Address of the <code>SD_File_t</code> variable used to manage the open file.
<code>path</code>	File path	Name or path of the file to open, for example <code>"data.csv"</code> .
<code>mode</code>	File mode	File access mode. This lab uses <code>SD_FILE_WRITE</code> .

Return value

`true` if the file is opened successfully; `false` if the file cannot be opened or created.

`SD_FILE_WRITE` is mapped to a file-creation mode for writing. If a file with the same name already exists, its previous contents are replaced. This behavior corresponds to the `FA_CREATE_ALWAYS` | `FA_WRITE` mode provided by FatFs [6] [4].

4.4 `SD_WriteLine()`**`SD_WriteLine`**

Writes a text string to the file and appends a line ending.

Syntax

```
SD_WriteLine(&file, text);
```

Parameters

Parameter	Type / accepted value	Description
<code>file</code>	SD File	File handle previously opened with <code>SD_Open()</code> .
<code>text</code>	Text	Text string to write to the file.

Return value

`true` if the write operation completes successfully; `false` if the file handle is invalid or the write operation fails.

In the program, each ADC sample is converted into one line of text before it is passed to `SD_WriteLine()` [6].

4.5 SD_Close() and SD_End()

`SD_Close()` ends the access session for an open file. After the file is closed successfully, the file handle no longer represents an open file. Closing the file is also a necessary step for completing filesystem data updates [4] [6] .

SD_Close

Closes the file managed by the `SD_File_t` handle.

Syntax

```
SD_Close(&file);
```

Parameters

Parameter	Type / accepted value	Description
<code>file</code>	SD File	File handle to close.

Return value

`true` if the file is closed successfully; `false` if the file handle is invalid or the close operation fails.

After the file has been closed, `SD_End()` releases any remaining filesystem resources, unmounts the filesystem, and stops SD interface operation in EduFramework [6] .

```
SD_Close(&file);  
SD_End();
```

5 Lab Exercise

5.1 Requirements

Build a data logger that uses a potentiometer as the analog data source. The program reads `ADC0_SE12` and stores ten samples in `data.csv` on the SD Card. Two consecutive samples are separated by `1000 ms` .

The program follows this sequence:

`Initialize SD` → `Open File` → `Acquire 10 ADC Samples` → `Write File` → `Close File` .

Serial Monitor is used to display the sample number and ADC value while the program is running. After completion, the SD Card can be removed and checked on a computer.

5.2 Program

In `src/main.c` , implement the program that has been verified on the hardware:

```
#include "Arduino.h"
#include "sd_card.h"
#include <stdio.h>
int main(void)
{
    SD_File_t file = {SD_INVALID_HANDLE};
    int adcValue = 0;
    uint8_t sample = 0U;
    char line[32];
    setup();
    Serial1_begin(9600U);
    if (false == SD_Begin(GPI04))
    {
        Serial1_println("SD initialization failed.");
        while (1)
        {
        }
    }
    if (false == SD_Open(&file, "data.csv", SD_FILE_WRITE))
    {
        Serial1_println("File open failed.");
        while (1)
        {
        }
    }
    SD_Writeline(&file, "sample,adc_value");
    for (sample = 1U; sample <= 10U; sample++)
    {
        adcValue = analogRead(ADC0_SE12);
        snprintf(
            line,
            sizeof(line),
            "%u,%d",
            (unsigned int)sample,
            adcValue);
        SD_Writeline(&file, line);
        Serial1_print("Sample ");
        Serial1_printInt(sample);
        Serial1_print(" | ADC: ");
        Serial1_printlnInt(adcValue);
        delay(1000U);
    }
    SD_Close(&file);
    SD_End();
    Serial1_println("Logging complete.");
    while (1)
    {
    }
    return 0;
}
```

Listing 17.5.1. Program for logging ADC data to an SD Card

`setup()` initializes the core EduFramework components, and `Serial1_begin(9600U)` prepares Serial Monitor. `SD_Begin(GPI04)` then initializes the SD Card and mounts the filesystem. If initialization fails, the program remains in the error loop instead of continuing with a storage device that is not ready [6].

`SD_Open()` creates `data.csv` using `SD_FILE_WRITE`. The first line is written with `SD_WriteLine()` to describe the two values stored in each sample. Inside the `for` loop, `analogRead(ADC0_SE12)` acquires the raw ADC value from the potentiometer. `snprintf()` creates a text string containing the sample number and ADC value; the string is then written to the file using `SD_WriteLine()`.

Each sample is separated by `1000 ms`. After ten samples, `SD_Close()` closes the file and `SD_End()` ends the SD Card session. The program prints `Logging complete.` only after these two operations, so the message acts as the indicator that the logging sequence has finished.

Application data flow:

Potentiometer → `analogRead()` → ADC value → `SD_WriteLine()` → SD Card.

5.3 Verification

Build and upload the program to the MaaZEDU Development Board, then open Serial Monitor at `9600` baud. During the approximately ten-second sampling interval, rotate the potentiometer to produce different voltage levels at `ADC0_SE12`.

Serial Monitor should show ten samples and end with:

```
Sample 1 | ADC: 4095
Sample 2 | ADC: 4095
Sample 3 | ADC: 1958
...
Sample 10 | ADC: 2868
Logging complete.
```

After `Logging complete.` appears, remove the SD Card and open `data.csv` on a computer. One result verified on the hardware is:

```
sample,adc_value
1,4095
2,4095
3,1958
4,1287
5,748
6,0
7,0
8,766
9,1654
10,2868
```

The exact ADC values depend on the potentiometer position. Verification does not require reproducing the exact numbers shown above; the goal is to confirm that the file is created successfully, contains ten samples, and changes when the potentiometer position changes.

EXPECTED RESULT

The SD Card is initialized successfully and `data.csv` is created. The program records ten ADC samples at `1000 ms` intervals, then closes the file and ends the SD Device session. The data stored in the file should correspond to the samples shown on Serial Monitor and vary with the potentiometer position.

6 Extension

The main exercise uses `SD_FILE_WRITE` so that each run creates a new dataset. The SD Card Device also provides APIs for appending data, reading files, synchronizing pending data, and retrieving storage information [6].

API / Mode	Purpose
<code>SD_FILE_APPEND</code>	Opens a file for appending at the end; the file is created if it does not already exist.
<code>SD_Flush()</code>	Synchronizes pending file data to the storage device without closing the file.
<code>SD_Read()</code>	Reads a specified number of bytes from an open file into an application buffer.
<code>SD_Exists()</code>	Checks whether a file or directory exists at the specified path.
<code>SD_GetCapacity()</code>	Retrieves the total and available filesystem capacity.

Table 17.6.1. Selected advanced functions of the SD Card Device

`SD_FILE_APPEND` differs from `SD_FILE_WRITE`: new data is written at the end of the file instead of replacing its previous contents. In EduFramework, these two modes are mapped to `FA_OPEN_APPEND` | `FA_WRITE` and `FA_CREATE_ALWAYS` | `FA_WRITE`, respectively [4] [6].

Note

`SD_Flush()` is useful when a file must remain open for a long period but the application wants to synchronize already written data to storage. It does not replace `SD_Close()` when the file access session has actually finished.

6.1 Extension Exercise

Replace `SD_FILE_WRITE` with `SD_FILE_APPEND` and modify the program so that each time the board is reset, ten new samples are appended to the end of the file instead of deleting data from the previous run. After at least two runs, inspect the file on a computer and confirm that data from both sessions is retained. Another extension is to replace the potentiometer with a data source used in a previous lab, such as temperature from an NTC thermistor or motion data from the MPU6050. The storage structure introduced in Lab 17 can then be reused without changing the SD Card management mechanism.

7 References

- [1] Youcef Bouchekioua, Hiroshi Matsui, Shigeru Watanabe, *A Versatile and Fast-Sampling Rate Wearable Analog Data Logger*, MethodsX, Vol. 10, 102098, 2023.
- [2] Patricia A. Beddows, Edward K. Mallon, *Cave Pearl Data Logger: A Flexible Arduino-Based Logging Platform for Long-Term Monitoring in Harsh Environments*, Sensors, Vol. 18, No. 2, 530, 2018.
- [3] SD Association, *Physical Layer Simplified Specification*, SD Specifications - Part 1 Simplified, Rev. 9.10, 2023.
- [4] ChaN, *FatFs - Generic FAT Filesystem Module*, FatFs Documentation.
- [5] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet*, S32K1XX, Rev. 15, 2026.
- [6] EduFramework, *SD Card Device API*, EduFramework v3.0.0 Source Code.

[7] MaaZEDU, *MaaZEDU Development Board Guide* .